Experiment-10

Aim: Implement Graph Traversals

Definition:

A **graph** is a data structure consisting of **vertices (nodes)** and **edges (connections)** between them.

- **Directed Graph (Digraph):** Edges have direction
- **Undirected Graph:** Edges have no direction
- **Weighted Graph:** Edges have weights

2. Graph Traversals

Graph traversal is visiting all nodes **in a systematic way**.

Types:

1. **Breadth-First Search (BFS):**
   - Visits nodes **level by level**
   - Uses a **queue**
2. **Depth-First Search (DFS):**
   - Visits nodes by going **deep along a branch before backtracking**
   - Uses **recursion** or a **stack**

BFS Algorithm:

1. Start from a chosen source node.
2. Create a queue and enqueue the source node.
3. Mark the node as visited.
4. While queue is not empty:
   - Dequeue a node v
   - Visit v
   - Enqueue all **unvisited adjacent nodes** and mark them as visited

DFS Algorithm (Recursive)

Algorithm:

1. Start from a chosen source node.
2. Mark the node as visited.
3. Visit all **unvisited adjacent nodes recursively**

Program:
from collections import deque

Dept of AIML

```
class Graph:
    def __init__(self, vertices):
        self.V = vertices
        self.adj = [[] for _ in range(vertices)]

    # Add edge
    def add_edge(self, u, v):
        self.adj[u].append(v)
        self.adj[v].append(u)  # for undirected graph

    # BFS traversal
    def bfs(self, start):
        visited = [False] * self.V
        queue = deque()
        visited[start] = True
        queue.append(start)

        print("BFS Traversal:", end=" ")
        while queue:
            node = queue.popleft()
            print(node, end=" ")
            for neighbor in self.adj[node]:
                if not visited[neighbor]:
                    visited[neighbor] = True
                    queue.append(neighbor)
        print()

    # DFS traversal (recursive)
    def dfs_util(self, node, visited):
        visited[node] = True
        print(node, end=" ")
        for neighbor in self.adj[node]:
            if not visited[neighbor]:
                self.dfs_util(neighbor, visited)

    def dfs(self, start):
        visited = [False] * self.V
        print("DFS Traversal:", end=" ")
        self.dfs_util(start, visited)
        print()

# Example usage
g = Graph(6)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 3)
```

Dept of AIML

```
g.add_edge(1, 4)
g.add_edge(2, 4)
g.add_edge(3, 5)
g.add_edge(4, 5)

g.bfs(0)
g.dfs(0)
```

OUTPUT:
BFS Traversal: 0 1 2 3 4 5
DFS Traversal: 0 1 3 5 4 2

Dept of AIML